Ingeniería de sistemas

Docente

Leidy Mariani Dorado Flórez

Materia

Sistemas distribuidos

Título

Práctica 4: Gestión de Imágenes y contenedores

Integrantes

Nasly Stefania Collazos

Valentina Samboní Dorado

Keny Stefany Quiñonez Campo

Juan Camilo Herrera Torres

Popayán, Cauca, 4 de Marzo de 2026

Práctica 4: Gestión de imágenes y contenedores

Gestión de Imágenes y Contenedores en Docker

📅 Semana: 4

👩‍🏫 Docente: Leidy Mariani Dorado

⏱ Duración: 2 horas

👥 Modalidad: Trabajo en equipos

🎯 Objetivo de la sesión

Al finalizar la práctica, el estudiante será capaz de:

- Diferenciar una imagen de un contenedor.
- Descargar imágenes desde Docker Hub.
- Crear, detener, reiniciar y eliminar contenedores.
- Analizar el consumo y el estado de los contenedores.
- Comprender la relación entre contenedores y sistemas distribuidos.

🔹 PARTE 1 – Exploración de Imágenes

✅ Actividad 1: Ver imágenes locales

Ejecutar:

docker images

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/u8lb548275606vccl8tb235mk
PS C:\Users\juanc\Downloads\clase6> docker images

IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest     ef54e839ef54       25.9kB         9.52kB
mi_sitio_web:latest    83b06e825d8b        237MB         62.9MB    U
mysql:latest           e5dc14f6e01c       1.27GB          283MB    U
nginx:latest           0236ee02dcbc        240MB         65.8MB    U
sitio_web2:latest      d273d4ebff0b        237MB         62.9MB    U
PS C:\Users\juanc\Downloads\clase6>
```

Respondan en equipo:

- ¿Qué es el IMAGE ID?

  Una imagen es como una plantilla o paquete que engloba y contiene todo lo necesario para la ejecución de una aplicación y su id es el identificador único de la imagen.

- ¿Qué significa TAG?

  Etiqueta que indica la versión de la imagen, se escribe después del nombre de la imagen separado por “:”, cuando no se especifica se asigna automáticamente el “latest”, esto no significa necesariamente que es la versión más reciente.

- ¿Qué representa SIZE?

  Es el tamaño-espacio que ocupa la imagen de docker, disk usage es la cantidad de espacio real consumido en disco y content size es el tamaño del contenido comprimido, por eso en mysql aparece 1.27 gb en disco pero solo 283 mb de contenido.

- ¿Una imagen se está ejecutando?

  Una imagen no se puede ejecutar por sí sola, lo que se ejecuta es un contenedor, que es una instancia de esa imagen.

✅ Actividad 2: Descargar nuevas imágenes

Cada equipo debe descargar una imagen diferente:

docker pull redis

O también pueden usar:

docker pull mongo
docker pull httpd
docker pull postgres

Redis

```
PS C:\WINDOWS\system32> docker pull redis
Using default tag: latest
latest: Pulling from library/redis
f25f5c85b90d: Pull complete
89b1b12d7109: Pull complete
4f4fb700ef54: Pull complete
141e1fc22067: Pull complete
ae77945b0371: Pull complete
214ecf6e5c87: Pull complete
e798a26a6510: Download complete
4a4db1a35d5b: Download complete
Digest: sha256:1c054d54ecd1597bba52f4304bca5afbc5565ebe614c5b3d7dc5b7f8a0cd768d
Status: Downloaded newer image for redis:latest
docker.io/library/redis:latest
PS C:\WINDOWS\system32>
```

Luego, ejecuten nuevamente:

docker images

```
PS C:\WINDOWS\system32> docker images

IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest     7af5f5c1c1c0       205MB         49.9MB    U
docker:latest            1ba18449911d       544MB          145MB    U
hello-world:latest       ef54e839ef54      25.9kB         9.52kB
mi_sitio_web:latest      83b06e825d8b       237MB         62.9MB    U
mysql:latest             e5dc14f6e01c      1.27GB          283MB    U
nginx:latest             0236ee02dcbc       240MB         65.8MB    U
postgres:latest          69e8582b781c       649MB          168MB
redis:latest             1c054d54ecd1       204MB         55.3MB
sitio_web2:latest        d273d4ebff0b       237MB         62.9MB    U
sitio_web:latest         d81cb3beaeb4       237MB         62.9MB    U
```

Reflexionar:

- ¿Qué cambió?

  Se descargó la imagen de redis y quedó guardada localmente con un id, ocupando 204 mb de disco y 55.3 mb de contenido.

- ¿El servicio ya está funcionando?

  Aún no ya que es solo una imagen, tendría que crearse y correr un contenedor a partir de esta imagen.

🔹 PARTE 2 – Gestión de Contenedores

✅ Actividad 3: Crear un contenedor

Ejemplo con nginx:

docker run-d--name servidor-web-p8081:80 nginx

```
PS C:\WINDOWS\system32> docker run --name mi_sitio_web -d -p 8081:80 nginx
0b93ade641713646c659f8922448988b7457216bb6158bc7f6229fbbc059edd6
```

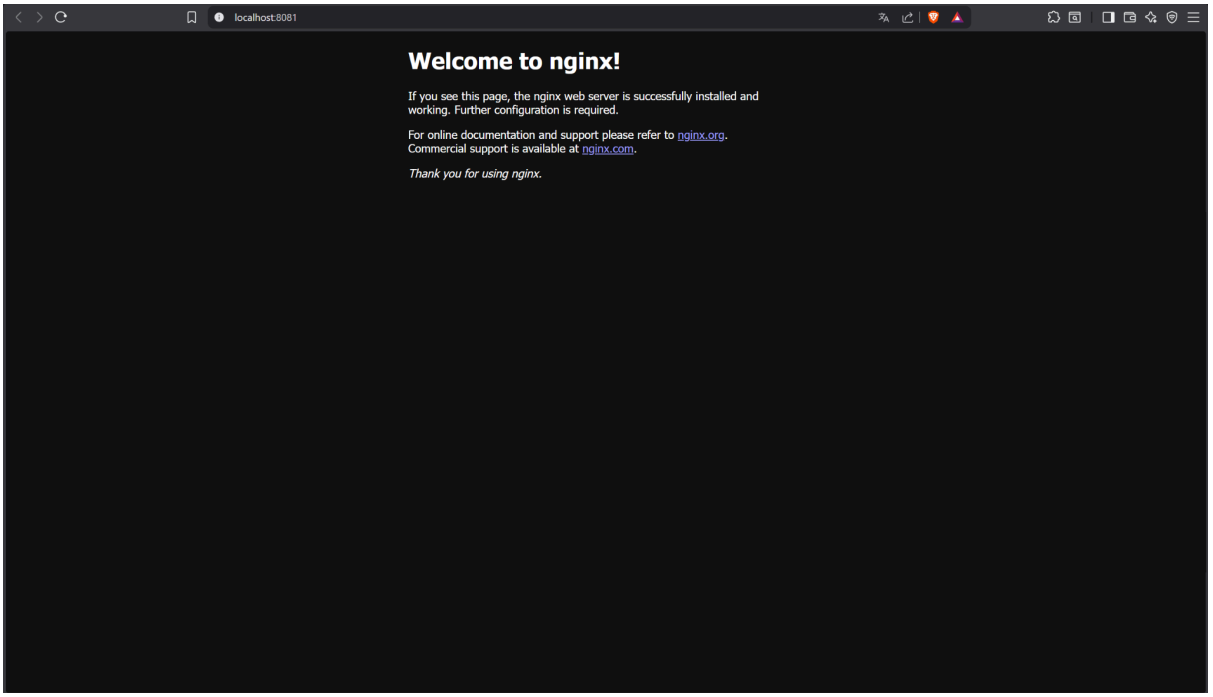
Verificar:

docker ps

```
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS          PORTS
    NAMES
0b93ade64171   nginx     "/docker-entrypoint.…"   2 seconds ago   Up 18 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tc
p   mi_sitio_web
PS C:\WINDOWS\system32>
```

Abrir el navegador:

<http://localhost:8081>



✅ Actividad 4: Administrar el contenedor

Detener:

dockerstop servidor-web

```
PS C:\WINDOWS\system32> docker stop mi_sitio_web
mi_sitio_web
PS C:\WINDOWS\system32>
```

Ver todos los contenedores:

docker ps-a

```
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED              STATUS                       PORTS
                 NAMES
0b93ade64171   nginx                  "/docker-entrypoint.…"   About a minute ago   Exited (0) 27 seconds ago
                 mi_sitio_web
de218807d6a4   docker                 "dockerd-entrypoint.…"   7 minutes ago        Exited (127) 7 minutes ago
                 relaxed_yonath
a90e3197d40d   api-node-demo          "docker-entrypoint.s…"   26 hours ago         Exited (255) 2 hours ago     0.0.0.0:
3004->3004/tcp   boring_northcutt
dc9a5977a965   mi_sitio_web:latest    "/docker-entrypoint.…"   2 days ago           Exited (255) 28 hours ago    80/tcp
                 upbeat_turing
57d6a22e1758   nginx                  "/docker-entrypoint.…"   2 days ago           Exited (0) 4 minutes ago
                 compose-web-1
58a0aa81280e   mysql                  "docker-entrypoint.s…"   2 days ago           Exited (255) 28 hours ago    3306/tcp
, 33060/tcp      compose-db-1
PS C:\WINDOWS\system32>
```

Reiniciar:

docker start servidor-web

```
PS C:\WINDOWS\system32> docker start mi_sitio_web
mi_sitio_web
```

Eliminar:

docker rm-f servidor-web

```
PS C:\WINDOWS\system32> docker rm -f mi_sitio_webb
mi_sitio_webb
PS C:\WINDOWS\system32>
```

🔹 PARTE 3 – Análisis Conceptual

Completar esta tabla:

Concepto	Imagen	Contenedor
¿Es ejecutable?	No, es estático	Si, es la imagen corriendo
¿Ocupa recursos en ejecución?	No, solo espacio	Si, CPU, red y memoria
¿Puede modificarse?	No, inmutable	Si
¿Puede eliminarse?	Si, usando docker rmi	Si, usando docker rm

---

🔹 PARTE 4 – Reto en Equipo

Cada grupo debe:

1. Ejecutar un contenedor diferente.

   Postgres

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\system32> docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
1b030cb6bea0: Pull complete
e232269e9149: Pull complete
70f3de31936a: Pull complete
3859b312d253: Pull complete
e7dee01ee448: Pull complete
0480b5ae6db9: Pull complete
56a239f8dc77: Pull complete
967e70a4ad93: Pull complete
eea32e6f70e8: Pull complete
4e2113a73362: Pull complete
be4ad6e904ed: Pull complete
358db71ef364: Pull complete
90f0f713dfa1: Download complete
3facd37768c9: Download complete
Digest: sha256:69e8582b781cb44fa4557b98ed586fe68361e320d9b12f9707494335634f4f3d
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
PS C:\WINDOWS\system32>
```

2. Mostrar que está activo.

```
PS C:\WINDOWS\system32> docker images

IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest     7af5f5c1c1c0       205MB         49.9MB    U
hello-world:latest       ef54e839ef54      25.9kB         9.52kB
mi_sitio_web:latest      83b06e825d8b       237MB         62.9MB    U
mysql:latest             e5dc14f6e01c      1.27GB          283MB    U
nginx:latest             0236ee02dcbc       240MB         65.8MB    U
postgres:latest          69e8582b781c       649MB          168MB
sitio_web2:latest        d273d4ebff0b       237MB         62.9MB    U
sitio_web:latest         d81cb3beaeb4       237MB         62.9MB    U
PS C:\WINDOWS\system32>
```

```
PS C:\WINDOWS\system32> docker run -d --name practica4_SD -p 8084:80 postgres
30ae7b517e8b9964f622630a640d8de0ee9d19b64cec78112375df4b3cc61183
```

postgres	latest	69e8582b781c	8 days ago	649.38 MB	

```
PS C:\WINDOWS\system32> docker run -d --name practica4_SDis -p 8084:5432 -e POSTGRES_PASSWORD=1234 postgres
a1a2a67eef4cef3fcfcbef6741840a149d973f65559a1de4065990025b552bd5
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED                  STATUS         PORTS
                 NAMES
a1a2a67eef4c   postgres   "docker-entrypoint.s…"   Less than a second ago   Up 2 seconds   0.0.0.0:8084->5432/tcp, [::]:
8084->5432/tcp   practica4_SDis
PS C:\WINDOWS\system32>
```

3. Explicar:
    - ¿Qué tipo de servicio es?

      Es una base de datos relacional la cual almacena datos en disco por medio del uso de tablas, filas y columnas. Se utiliza para el almacenamiento de información estructurada y persistente como usuarios, pedidos o transacciones, como es una base de datos con información sensible, se solicita contraseña porque si se permite que cualquier usuario se conecte sin credenciales sería un riesgo grave.

    - ¿En qué tipo de sistema distribuido podría usarse?

      En sistemas distribuidos se usa como base de datos principal de uno o más microservicios, por ejemplo el servicio de usuarios tiene su propio PostgreSQL y el servicio de pedidos el suyo aparte, siguiendo la idea de que cada microservicio gestiona y almacena sus propios datos independientemente.

    - ¿Qué pasaría si ese servicio falla?

      Todos los datos de ese servicio quedarían inaccesibles constituyendo el escenario más crítico posible. Las operaciones de lectura y escritura se detienen y depende de la configuración podría haber pérdida de datos. Por eso desde producción debe configurarse réplicas y backups automáticos.

🔹 PARTE 5 – Análisis Crítico

Responder:

1. ¿Cuál es la diferencia real entre imagen y contenedor?

Se puede entender de tal manera que la imagen es el molde y el contenedor es el objeto creado a partir de ese molde, la imagen no cambia ni consume recursos al ser estática. El contenedor es la imagen corriendo, tiene su propio procesamiento. Se pueden crear diversos (muchos) contenedores distintos solo requiriendo una imagen y van a funcionar independientemente.

2. ¿Por qué Docker facilita la escalabilidad?

Lo facilita de tal manera que si se requiere que la aplicación que usas tiene un tráfico más alto de datos, en segundos se pueden levantar diversos contenedores de la misma imagen para ayudar. Los contenedores se crean completamente listos para trabajar sin tener que preocuparte por levantar y configurar un nuevo servidor y dependencias.

3. ¿Cómo se relaciona esto con microservicios?

Cada microservicio vive en su propio contenedor, piezas pequeñas, independientes y especializadas. La base de datos en un contenedor, las APIs en otro, el servidor web también aparte, precisamente aquí encontramos la relación con los microservicios, cada contenedor se puede actualizar, modificar, reiniciar o escalar independientemente sin afectar otro módulo del sistema ni detener funcionalidades, que es lo que justamente busca un arquitectura basada en microservicios.

4. ¿Qué ventajas ofrece frente a instalar software directamente?

- Se evitan conflictos entre programas
- Funciona en cualquier computador
- Instalación más rápida
- Facilidad en compartir aplicaciones
- Se pueden tener dos versiones distintas por ejemplo de Node o MySQL corriendo al mismo tiempo sin ningún problema.
- Si algo se rompe simplemente se elimina el contenedor y se crea uno nuevo sin rastros ni residuos en tu sistema

Conclusión grupal

El desarrollo de este taller permitió comprender de manera práctica el funcionamiento de Docker y su utilidad para ejecutar servicios dentro de contenedores de forma rápida y organizada. A través de los diferentes ejercicios se logró identificar la diferencia entre imágenes y contenedores, así como la forma de descargar, ejecutar y verificar servicios como PostgreSQL o Redis. Además, se evidenció cómo los contenedores facilitan la implementación de aplicaciones y servicios en diferentes entornos sin necesidad de configuraciones complejas. Esta experiencia permitió fortalecer los conocimientos sobre tecnologías utilizadas en sistemas distribuidos y su importancia para el despliegue eficiente de aplicaciones modernas.

📌 Entregable de la clase

Cada equipo debe subir al repositorio:

- Capturas de comandos ejecutados.
- Respuestas a las preguntas.
- Tabla comparativa completada.
- Conclusión grupal (mínimo 5 líneas).